

Design Templates

Generic Template — Format Guide & Test Harness

Every template file in this folder follows the same two-part structure:

- 1. **Template algorithm + template code** — the reusable pattern, with specific LeetCode problems used as concrete examples inside the code.
- 2. **Quick reference table + question/solution** — a table listing every problem (with a *Variation from Template* column), followed by each problem's full solution where deviations are marked `// ← VARIATION: explanation`.

Generic Test Harness

Use this boilerplate to run any single problem locally. Replace `data / solve()` with the problem's input and logic.

```
class Question {  
  
    private int data;  
  
    public Question(int data) {  
        this.data = data;  
    }  
  
    public void solve() {  
        // TODO  
    }  
}  
  
public class Solution {  
  
    public static void main(String[] args) {  
        Question q = new Question(0);  
        q.solve();  
    }  
}
```

Naming Conventions (enforced across all template files)

Context	Convention
Index variables	<code>i</code> , <code>j</code> , <code>k</code> (never <code>left</code> , <code>right</code> , <code>low</code> , <code>mid</code> , <code>high</code>)
Linked list	<code>sentinel</code> , <code>previous</code> , <code>current</code> (never <code>prev</code> , <code>cur</code> , <code>curr</code>)
String building	<code>StringBuilder builder = new StringBuilder();</code> (never <code>sb</code>)
Result accumulator	<code>result</code> (never <code>res</code>); use <code>output</code> if <code>result</code> is already taken in scope
Queue / deque	<code>Queue<Integer> queue = new ArrayDeque<>();</code> (never <code>dq</code>)
BFS over tree	<code>Queue<TreeNode> queue = new ArrayDeque<>();</code>
Grid directions	<code>int[] dr = {1,-1,0,0}, dc = {0,0,1,-1};</code> (DOWN, UP, RIGHT, LEFT); iterate <code>for (int dir=0; dir<4; dir++)</code> using <code>i+dr[dir]</code> , <code>j+dc[dir]</code>
Character frequency	<code>count[ch - 'a']++;</code>
Digit frequency	<code>count[ch - '0']++;</code>
Char to integer build	<code>num = num * 10 + (ch - '0');</code>

Template Index

File	Category	Core Pattern
binary_search.md	Binary Search	Closed <code>[i, j]</code> , half-open, minimize/maximize answer space
two_pointers.md	Two Pointers	Opposite ends, same-direction write pointer, Dutch flag
prefix_sum_hashmap.md	Prefix Sum	Count/length with HashMap, tree path sums
sorting.md	Sorting	Merge sort, quick sort, quick select, count-during-merge
sliding_window.md	Sliding Window	Fixed, max-variable, min-variable windows
linked_list.md	Linked List	Delete/filter, reversal, fast/slow, merge, composite
dfs.md	DFS / Backtracking	Tree return-up/pass-down, grid DFS, graph 3-color, backtracking, BST
bfs.md	BFS	Level-order with size snapshot, position indexing
graph.md	Graph	BFS shortest path, topo sort, union find, Dijkstra, bipartite, Prim's MST
dynamic_programming.md	Dynamic Programming	1D/2D/grid/interval DP, knapsack, state machine
stack_queue.md	Stack / Queue / Heap	Monotone stack/deque, two heaps, priority queue
greedy.md	Greedy	Sort-by-end, sort-by-start, sort + heap, non-interval
string.md	String	Frequency counting, hashing, bucket sort, expand-from-center
bit_manipulation.md	Bit Manipulation	XOR cancel, mod-k state machine, bitmask as set, power checks
trie.md	Trie	Insert/search/startsWith, wildcard DFS, grid pruning
design.md	Design	HashMap + linked list (LRU), frequency maps (LFU), array + map
math.md	Math	Fast power, base conversion, sieve, digit manipulation

Quick Reference Table

#	Name	Description	Algorithm	Variation from Template	Time	Space
146	LRU Cache	O(1) get/put with LRU eviction	HashMap + doubly linked list; move accessed node to front	Standard	O(1) get/put	O(capacity)
460	LFU Cache	O(1) get/put with LFU eviction	HashMap<key→val>, HashMap<key→freq>, HashMap<freq→LinkedHashSet>; track minFreq	LFU : extra frequency map + minFreq variable	O(1) all ops	O(capacity)
380	Insert Delete GetRandom O(1)	Insert, remove, getRandom all in O(1)	HashMap<val→index> + ArrayList; swap-with-last on delete	Swap-to-end : on remove, swap target with last element to maintain dense array	O(1) all ops	O(n)

Canonical Template — HashMap + Doubly Linked List (LRU)

```
// MENTAL MODEL: HashMap gives O(1) lookup; the doubly linked list orders by recency so the LRU is always one hop from tail.
// WHEN: "O(1) get/put with eviction by recency or frequency"

// CacheNode for doubly linked list (named CacheNode to distinguish from ListNode contexts)
class CacheNode {
    int key, value;
    CacheNode previous, next;
    CacheNode(int key, int value) { this.key = key; this.value = value; }
}

// — SENTINEL NODES — head (MRU side) ↔ ... ↔ tail (LRU side)
CacheNode head = new CacheNode(0, 0), tail = new CacheNode(0, 0);
// Initialize: head <-> tail
head.next = tail; tail.previous = head;

// — REMOVE NODE —
void remove(CacheNode node) {
    node.previous.next = node.next;
    node.next.previous = node.previous;
}

// — INSERT AFTER HEAD (= mark as MRU) —
void insertAfterHead(CacheNode node) {
    node.next = head.next;
    node.previous = head;
    head.next.previous = node;
    head.next = node;
}

// — LRU EVICTION — tail.previous is LRU
CacheNode lru = tail.previous;
remove(lru);
map.remove(lru.key);
```

Variations

```
// VARIATION 1: LFU — adds a frequency dimension
// Need: key->value, key->freq, freq->orderedSet<key>, and minFreq
// When a key is accessed: freq++, move from freqToKeys[oldFreq] to freqToKeys[newFreq]
// On eviction: remove first element from freqToKeys[minFreq]
// LinkedHashSet preserves insertion order → oldest at front for same frequency

// VARIATION 2: Insert Delete GetRandom — use array for O(1) random
// ArrayList stores values; HashMap stores val->index in array
// On remove: swap target with last element, update map, remove last
// On getRandom: return list.get(random.nextInt(list.size()))
```

Part 1 — LRU Cache

#146 LRU Cache

Description: Design a data structure that follows the LRU (Least Recently Used) eviction policy. `get(key)` returns the value or -1. `put(key, value)` inserts/updates; evicts LRU when over capacity. Both operations must be O(1).

Algorithm: Combine a HashMap (O(1) key lookup) with a doubly linked list (O(1) insertion/deletion). The list order represents recency — most recently used at head, least recently used at tail. On every `get` or `put`, move the accessed node to the front.

Intuition: the HashMap answers "where is this key" instantly while the linked list keeps everything ordered by recency, so the victim to evict is always the node next to the tail.

```
class LRUCache {
    private final int capacity;
    private final Map<Integer, CacheNode> map = new HashMap<>();
    private final CacheNode head = new CacheNode(0, 0), tail = new CacheNode(0, 0);

    public LRUCache(int capacity) {
        this.capacity = capacity;
        head.next = tail;
        tail.previous = head;
    }

    public int get(int key) {
        if (!map.containsKey(key)) return -1;
        CacheNode node = map.get(key);
        remove(node);
        insertAfterHead(node);    // ← move to MRU position
        return node.value;
    }

    public void put(int key, int value) {
        if (map.containsKey(key)) {
            CacheNode node = map.get(key);
            node.value = value;
            remove(node);
            insertAfterHead(node);    // ← update and move to MRU position
        } else {
            if (map.size() == capacity) {
                CacheNode lru = tail.previous;    // ← LRU is just before tail sentinel
                remove(lru);
                map.remove(lru.key);
            }
            CacheNode node = new CacheNode(key, value);
            map.put(key, node);
            insertAfterHead(node);
        }
    }

    private void remove(CacheNode node) {
        node.previous.next = node.next;
        node.next.previous = node.previous;
    }

    private void insertAfterHead(CacheNode node) {
        node.next = head.next;
        node.previous = head;
        head.next.previous = node;
        head.next = node;
    }

    static class CacheNode {
        int key, value;
        CacheNode previous, next;
        CacheNode(int key, int value) { this.key = key; this.value = value; }
    }
}
```

Time $O(1)$ get/put | **Space** $O(\text{capacity})$

Part 2 — LFU Cache

#460 LFU Cache

Description: Design a data structure for a Least Frequently Used cache. Both `get` and `put` must be $O(1)$. On eviction, remove the least frequently used key; among ties in frequency, remove the least recently used.

Algorithm: Three maps: `keyToVal`, `keyToFreq`, `freqToKeys` (`freq` → `LinkedHashSet` for insertion-order LRU within same `freq`). Track `minFreq`. On access: increment `freq`, move key from old to new `freq` set; if old `freq` set is now empty and it was `minFreq`, increment `minFreq`.

Intuition: bucket keys by how often they're used; the eviction victim is always the oldest key in the lowest-frequency bucket, which `minFreq` points straight at.

```
class LFUCache {
    private final int capacity;
    private int minFreq = 0;
    private final Map<Integer, Integer> keyToVal = new HashMap<>();
    private final Map<Integer, Integer> keyToFreq = new HashMap<>();
    private final Map<Integer, LinkedHashSet<Integer>> freqToKeys = new HashMap<>();

    public LFUCache(int capacity) { this.capacity = capacity; }

    public int get(int key) {
        if (!keyToVal.containsKey(key)) return -1;
        increaseFreq(key);          // ← VARIATION: frequency tracking on every access
        return keyToVal.get(key);
    }

    public void put(int key, int value) {
        if (capacity <= 0) return;
        if (keyToVal.containsKey(key)) {
            keyToVal.put(key, value);
            increaseFreq(key);
        } else {
            if (keyToVal.size() >= capacity)
                removeMinFreqKey();    // ← VARIATION: evict by min frequency
            keyToVal.put(key, value);
            keyToFreq.put(key, 1);
            freqToKeys.computeIfAbsent(1, k -> new LinkedHashSet<>()).add(key);
            minFreq = 1;               // ← VARIATION: reset minFreq to 1 on new insert
        }
    }

    private void increaseFreq(int key) {
        int freq = keyToFreq.get(key);
        keyToFreq.put(key, freq + 1);
        freqToKeys.get(freq).remove(key);
        if (freqToKeys.get(freq).isEmpty()) {
            freqToKeys.remove(freq);
            if (minFreq == freq) minFreq++;    // ← VARIATION: update minFreq only if this was min
        }
        freqToKeys.computeIfAbsent(freq + 1, k -> new LinkedHashSet<>()).add(key);
    }

    private void removeMinFreqKey() {
        LinkedHashSet<Integer> keys = freqToKeys.get(minFreq);
        int evict = keys.iterator().next();    // ← LinkedHashSet: first = oldest at this freq
        keys.remove(evict);
        if (keys.isEmpty()) freqToKeys.remove(minFreq);
        keyToVal.remove(evict);
        keyToFreq.remove(evict);
    }
}
```

Time $O(1)$ all operations | **Space** $O(\text{capacity})$

Part 3 — Array + HashMap (Insert Delete GetRandom)

#380 Insert Delete GetRandom O(1)

Description: Design a set where `insert`, `remove`, and `getRandom` all run in $O(1)$ average time. `getRandom` returns any element with equal probability.

Algorithm: HashMap maps value→index in an ArrayList. On remove, swap the target with the last element (update the swapped element's index in the map), then remove the last. This keeps the array dense without shifting. `getRandom` uses `Random.nextInt(size)`.

Intuition: an array gives $O(1)$ random indexing but $O(n)$ middle-deletion; swapping the victim with the last element turns deletion into an $O(1)$ pop while staying dense.

```
class RandomizedSet {
    private final Map<Integer, Integer> map = new HashMap<>(); // val → index in list
    private final List<Integer> list = new ArrayList<>();
    private final Random rand = new Random();

    public boolean insert(int val) {
        if (map.containsKey(val)) return false;
        list.add(val);
        map.put(val, list.size() - 1);
        return true;
    }

    public boolean remove(int val) {
        if (!map.containsKey(val)) return false;
        int i = map.get(val);
        int last = list.get(list.size() - 1);
        list.set(i, last); // ← VARIATION: swap target with last element
        map.put(last, i); // ← VARIATION: update swapped element's index
        list.remove(list.size() - 1);
        map.remove(val);
        return true;
    }

    public int getRandom() {
        return list.get(rand.nextInt(list.size())); // ← VARIATION: O(1) random via ArrayList
    }
}
```

Time $O(1)$ all ops | **Space** $O(n)$

LRU vs LFU vs RandomizedSet — Pattern Chooser

Signal	Design
"evict least recently used"	LRU: HashMap + doubly linked list, move to head on access
"evict least frequently used"	LFU: 3 HashMaps + LinkedHashSet + <code>minFreq</code> tracking
" $O(1)$ insert/delete/random"	RandomizedSet: HashMap + ArrayList, swap-with-last on delete

Key Invariants

```
LRU: head.next = MRU, tail.previous = LRU
LFU: minFreq always points to the smallest existing frequency bucket
    LinkedHashSet preserves insertion order → iterator().next() = LRU within bucket
RandomizedSet: list[map[val]] == val at all times (after swap, update the map for the swapped key)
```